

■ Hoja de Referencia Rápida: Spring Framework (IoC & Inyección)

■ 1. Anotaciones de Componentes e Inyección (IoC)

Spring Framework se basa en **Inversión de Control (IoC)** e **Inyección de Dependencias (DI)** para conectar Beans dentro del ApplicationContext.

Anotación	Propósito	Ejemplo
<code>@Component</code>	Anotación genérica para registrar un Bean.	<code>@Component</code>
<code>@Service</code>	Especialización para lógica de negocio.	<code>@Service</code>
<code>@Repository</code>	Capa de acceso a datos y traducción de SQL.	<code>@Repository</code>
<code>@Controller</code>	Controlador Web MVC para vistas HTML.	<code>@Controller</code>
<code>@RestController</code>	Controlador REST con <code>@ResponseBody</code> .	<code>@RestController</code>
<code>@Configuration</code>	Clase contenedora de definiciones <code>@Bean</code> .	<code>@Configuration</code>

<code>@Bean</code>	Define un Bean sobre métodos de fábrica.	<code>@Bean</code>
<code>@Autowired</code>	Inyecta dependencia automáticamente por tipo.	<code>@Autowired</code>
<code>@Qualifier</code>	Desambigua entre múltiples Beans.	<code>@Qualifier("sms")</code>
<code>@Primary</code>	Establece el Bean preferido por defecto.	<code>@Primary</code>
<code>@Value</code>	Inyecta propiedades o expresiones SpEL.	<code>@Value("\${app.desc}")</code>
<code>@Lazy</code>	Retarda la inicialización del Bean.	<code>@Lazy</code>

■ ■ 2. Estilos de Inyección de Dependencias

A. Inyección por Constructor (Estándar Recomendado)

```
@Service
public class ClienteService {

    private final ClienteRepository clienteRepository;
    private final NotificadorService notificadorService;

    // En Spring 4.3+, @Autowired es implícito
    public ClienteService(ClienteRepository repo,
                          NotificadorService notificador) {
        this.clienteRepository = repo;
        this.notificadorService = notificador;
    }
}
```

```
}
```

B. Inyección por Campo (No Recomendada)

```
@Service
public class PedidoService {
    @Autowired // Inyeccion directa por campo
    private PedidoRepository pedidoRepository;
}
```

■ 3. Ciclo de Vida del Bean y Alcances (Scopes)

Alcance (Scope)	Descripción	Anotación
singleton	(Por defecto) Instancia única global.	@Scope("singleton")
prototype	Nueva instancia en cada petición.	@Scope("prototype")
request	Instancia por cada petición HTTP.	@RequestScope
session	Instancia por cada sesión HTTP.	@SessionScope

```
@Component
public class ConexionExternaService {
    @PostConstruct
    public void inicializar() {
        System.out.println("Conexion inicializada.");
    }
}
```

```
@PreDestroy
public void limpiar() {
    System.out.println("Conexion cerrada.");
}
}
```

■ 4. Expresiones SpEL (Spring Expression Language)

```
@Component
public class CalculadoraDescuento {

    @Value("#{systemProperties['user.language']}")
    private String idiomaSistema;

    @Value("#{tasaService.obtenerTasa() * 1.13}")
    private double tasaCalculada;

    @Value("#{pedido.monto > 50000 ? 0.10 : 0.05}")
    private double porcentajeDescuento;
}
```

■ 5. Perfiles de Entorno (@Profile)

```
@Configuration
@Profile("dev")
public class DevDatabaseConfig {

    @Bean
    public DataSource dataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .build();
    }
}
```